Why we can't just buy a *bigger machine*

you have spent the last two years writing programs that fit on one machine. You have learned how to do this more efficiently by multiprocessing in python and using GPUs in machine learning. Those are both single-machine answers to "my program is too slow." So before we start talking about clusters, lets ask the honest question. Why isn't a single, very fast machine enough?

Scale-up has a ceiling

The first answer most people reach for is **scale-up**: buy a bigger machine. More cores, more RAM, faster disks, faster network card. This works for a while. The reason it stops working is not philosophical. It is that the cost curve goes vertical.

A respectable workstation with 32 cores and 128 GB of RAM lives in the low-thousands-of-dollars range. A two-socket server with 96 cores and 1 TB of RAM is in the tens of thousands. A four-socket monster with 384 cores and 6 TB of RAM is well into the six figures, and that's before the support contract. The hardware exists. The dollars-per-byte and dollars-per-core curves both turn against you sharply past a certain point, because you are paying a premium for the engineering effort of cramming more silicon into a single box.

The second answer is **scale-out**: buy more boxes. Each box is unremarkable on its own, but you stitch them together with the network and treat them as one logical machine. The cost curve here is approximately linear. Twice as much aggregate storage costs roughly twice as much money. Twice as much aggregate CPU costs roughly twice as much money. There is overhead and there are coordination costs, but the curve does not go vertical.

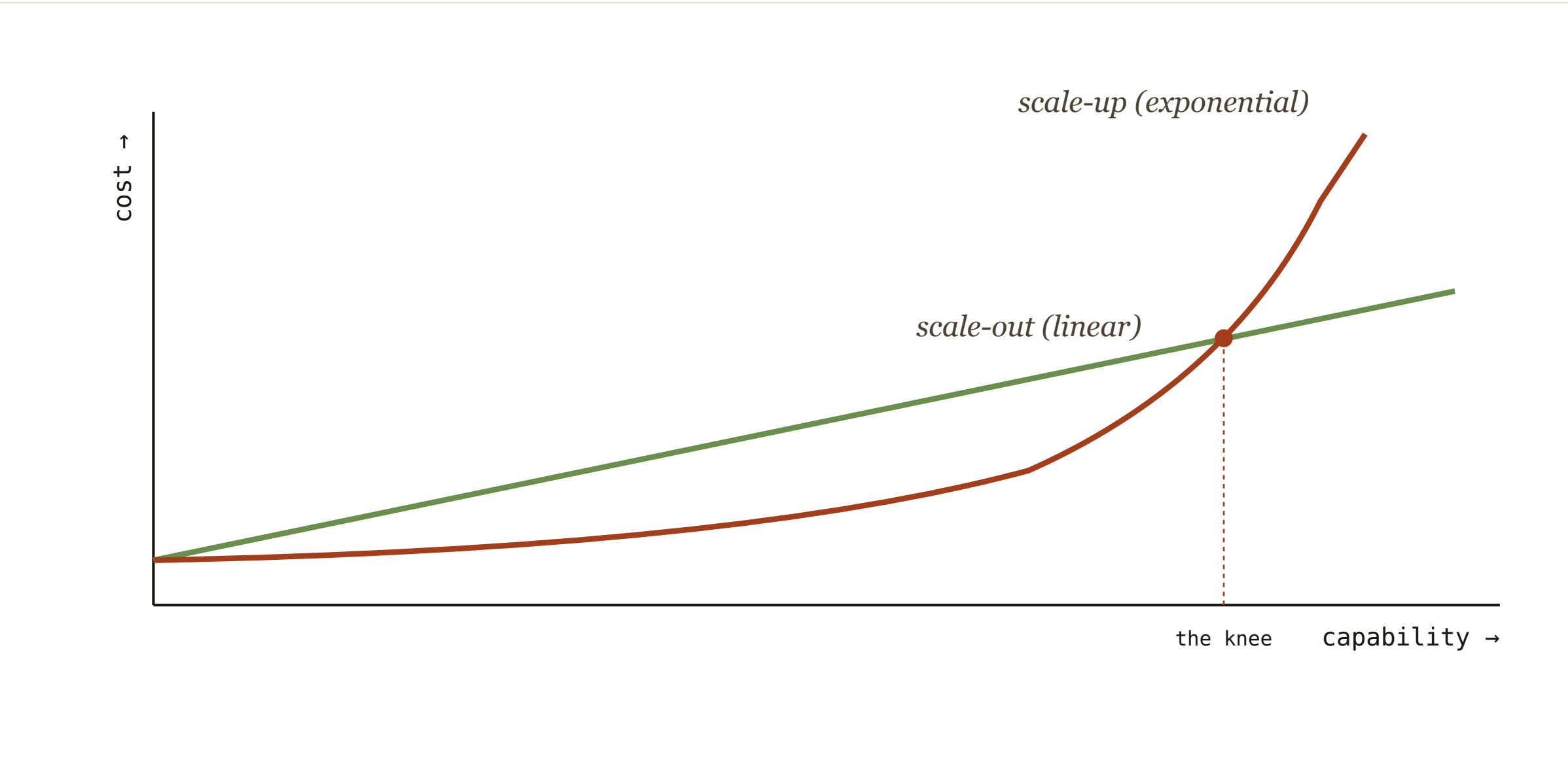

FIG 1.1 *The cost curve for scale-up bends upward; the cost curve for scale-out stays approximately linear. The "knee" is the inflection point past which buying another box is cheaper than upgrading the box you have.*

So scale-out wins on cost at the scales we care about. That, however, is not the most interesting reason to use a cluster.

Failure is the common case

A single machine fails rarely. Mean time between failures for a good server is on the order of years. When it fails we shut things down, fix it, bring it back. Operationally this is fine, because the failure is rare and the system is one thing.

Now stand up 1,000 of those servers. Each one still fails rarely (years between failures), but the rate at which *something somewhere in your fleet* is failing is now measured in hours. With 10,000 servers you are looking at multiple failures per day as the steady state. Disk failures, NIC failures, RAM bit flips, switches going down, fans dying, software crashes, kernel panics, kernel updates that didn't take, a backhoe cutting a fiber link in the parking lot, a rack losing power because a fuse blew.

This is the most important reframing in this entire course. A distributed system is not "a single machine, but bigger." It is a fundamentally different operational regime in which partial failure is happening continuously, and the system's job is to keep working anyway. **Failure stops being an event and starts being the weather.**

If your design treats failure as an exceptional case to be caught and reported, your design is wrong for this scale. Failure is the normal case. The exceptional case is the brief window in which nothing is broken.

The fallacies of distributed computing

The fallacies of distributed computing are eight assumptions that single-machine programmers carry over by reflex and that all turn out to be wrong on a network. The list was assembled by L. Peter Deutsch and others at Sun Microsystems in 1994. Every one of them is going to bite us during this course. Lets walk through them.

1. **The network is reliable.** It isn't. Packets get dropped, connections time out, NICs flake, switches reboot. Every network call has some non-zero probability of failing.
2. **Latency is zero.** It isn't, and it's worse than you think. The speed of light through fiber across a continent imposes a hard floor of tens of milliseconds. Even within a datacenter, a round trip is hundreds of microseconds, which is astronomic compared to memory access.
3. **Bandwidth is infinite.** It isn't. The link between two racks is sized for the average case, and any analytic workload tries hard to saturate it.
4. **The network is secure.** It isn't. Anything on the wire is on the wire.
5. **Topology doesn't change.** It does. Machines move racks, IPs get reassigned, services migrate, DNS records change.
6. **There is one administrator.** There isn't, and they don't agree. Different teams own different layers, and the team that runs Spark is not the team that runs the network switches.
7. **Transport cost is zero.** It isn't. Serializing a Python object to bytes, sending it across the wire, and deserializing on the other side is real CPU work and not negligible.
8. **The network is homogeneous.** It isn't. Some hops are 10 Gbps, some are 25, some are 100; some are within a rack and some are across a WAN.

You should read each of these as "the thing single-machine programmers get to assume, but you don't." This unit directly attacks fallacies 1, 2, and 5. Units 2 and 3 attack fallacies 3 and 7. We will run into every single one of them by the end of the semester.

The numbers we are working against

Jeff Dean's *Numbers Every Programmer Should Know* is fundamental knowledge for this course and should be a review from SD212. Internalize the rough orders of magnitude. The ones that bind most often in big-data work:

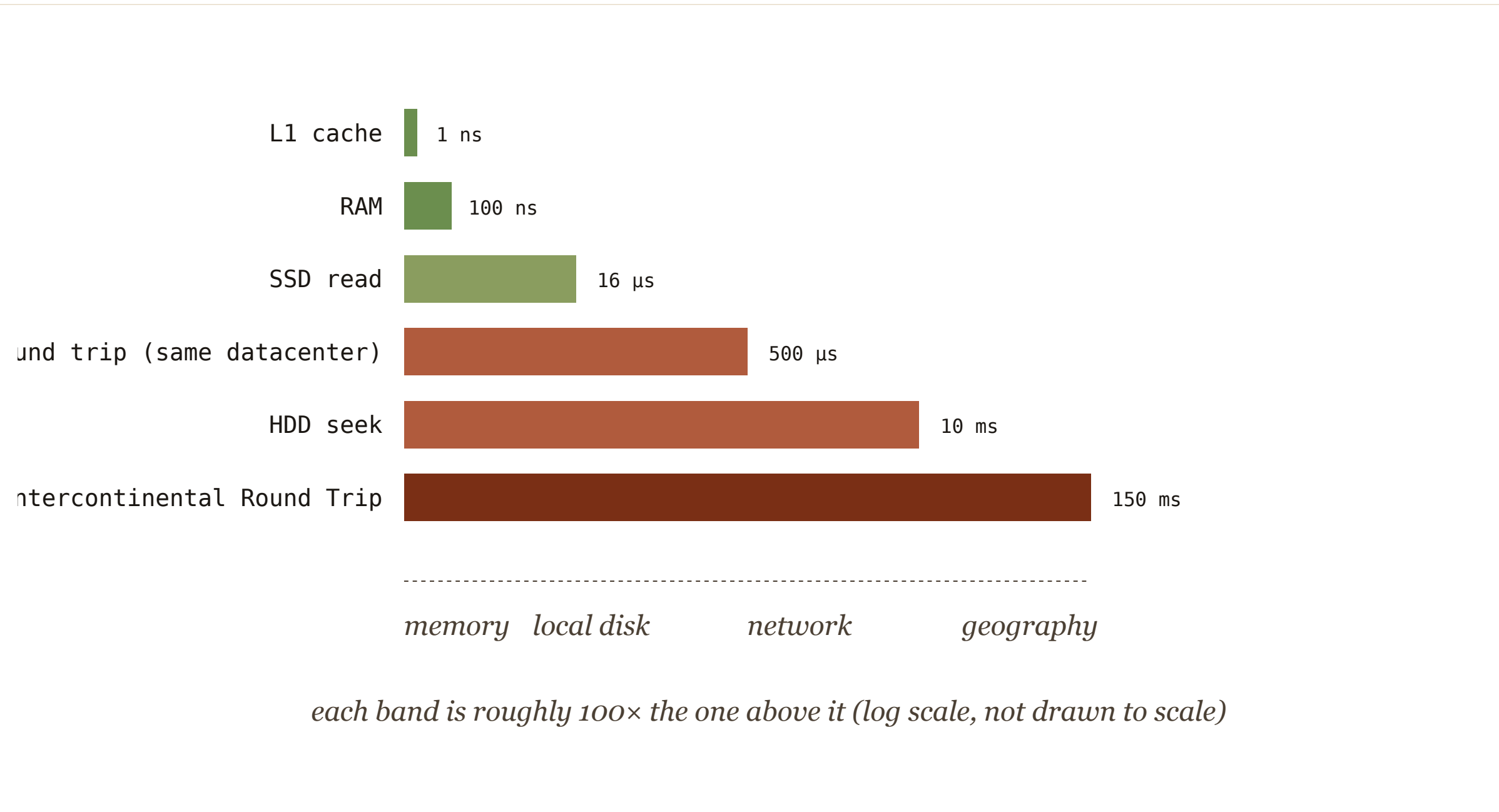

FIG 1.2 *The latency hierarchy. The gap between RAM and disk is enormous, which is why caching matters. The gap between RAM and the network is also enormous, which is why "just send it over the network" is never free.*

Two takeaways. First, the gap between memory and disk is huge, which is why caching matters. Second, the gap between memory and network is also huge, which is why "just send it over the network" is never free. We will see this in Unit 2 when we talk about shuffles. A shuffle is, fundamentally, the act of choosing to pay the network-roundtrip number millions of times in a single query. The whole reason shuffles are expensive is hiding in this diagram.

Latency vs throughput

One last distinction before we move on, because students confuse the two and the confusion is expensive.

Latency is how long a single operation takes from start to finish. **Throughput** is how many operations complete per unit time. They are related but not interchangeable.

A 10 Gbps network link has a throughput of about 1.25 GB/s in each direction. That is a lot of bytes. But the latency of sending a one-byte message across that link is still hundreds of microseconds, because most of that time is propagation delay and software overhead, not transmission time. If your job needs to send a billion small messages, you are gated by latency × billion. If your job needs to send one billion-byte message, you are gated by total-bytes / bandwidth.

This is why batching is the most important performance optimization in distributed systems. Sending 1,000 records as 1 batch of 1,000 records is dramatically faster than sending 1,000 records as 1,000 single-record messages, even though the total number of bytes is roughly the same. We will use this principle dozens of times.

So why distributed?

We need a cluster for two reasons:

1. Our data does not fit on one machine, and getting it to fit would cost more than buying ten machines.
2. The cluster is *less* fragile than a single machine, because we can design it so that no single node failing brings it down.

That second reason takes some work to actually achieve. We need a deliberate design that turns a pile of unreliable nodes into a reliable system. The whole rest of this unit is about how. We will pick up the storage half of that story in next time by deriving the design principles from first principles, and then looking at HDFS as the cleanest worked example.

Partitioning, replication, and the design of *HDFS*

We have established that we want a cluster and that the cluster must tolerate the failure of any single node. Lets derive the two foundational principles of distributed storage from scratch before we look at any specific system. Both principles will reappear in every system in this course, from HDFS to S3 to Kafka to Spark.

Data for this unit

We will use **MLB Statcast** for this unit. Every pitch thrown in MLB since 2015 is captured at roughly one-millisecond resolution with the full physics of the pitch: release point, spin axis, break, launch angle, exit velocity, and around eighty other columns. One season is roughly 700,000 pitches; ten seasons is in the seven-million-row neighborhood. We will store this dataset six different ways before the unit is over and measure each one. By the time we reach Spark in Unit 2, the question "what file format should this be in" will not feel abstract.

The first problem: the data doesn't fit

You have a 4 TB Statcast dataset. You have ten machines, each with a 1 TB disk. The dataset does not fit on any one machine, and you would like it to still feel like one logical dataset when you read it. What do you do?

Think of eating a pizza. A large pizza is cumbersome to eat as one piece, so common practice is to cut it into slices. Each slice is a smaller subset of the pizza, and if you collect all the slices you recover the whole. The slice serves a purpose: it makes the pizza manageable. This is the principle of **partitioning**, and a slice of a dataset is called a **partition**.

To store our 4 TB dataset on ten 1 TB machines, we partition it into ten roughly-equal pieces and place one partition on each machine. The dataset still exists logically as one dataset; we have just spread it physically across the cluster.

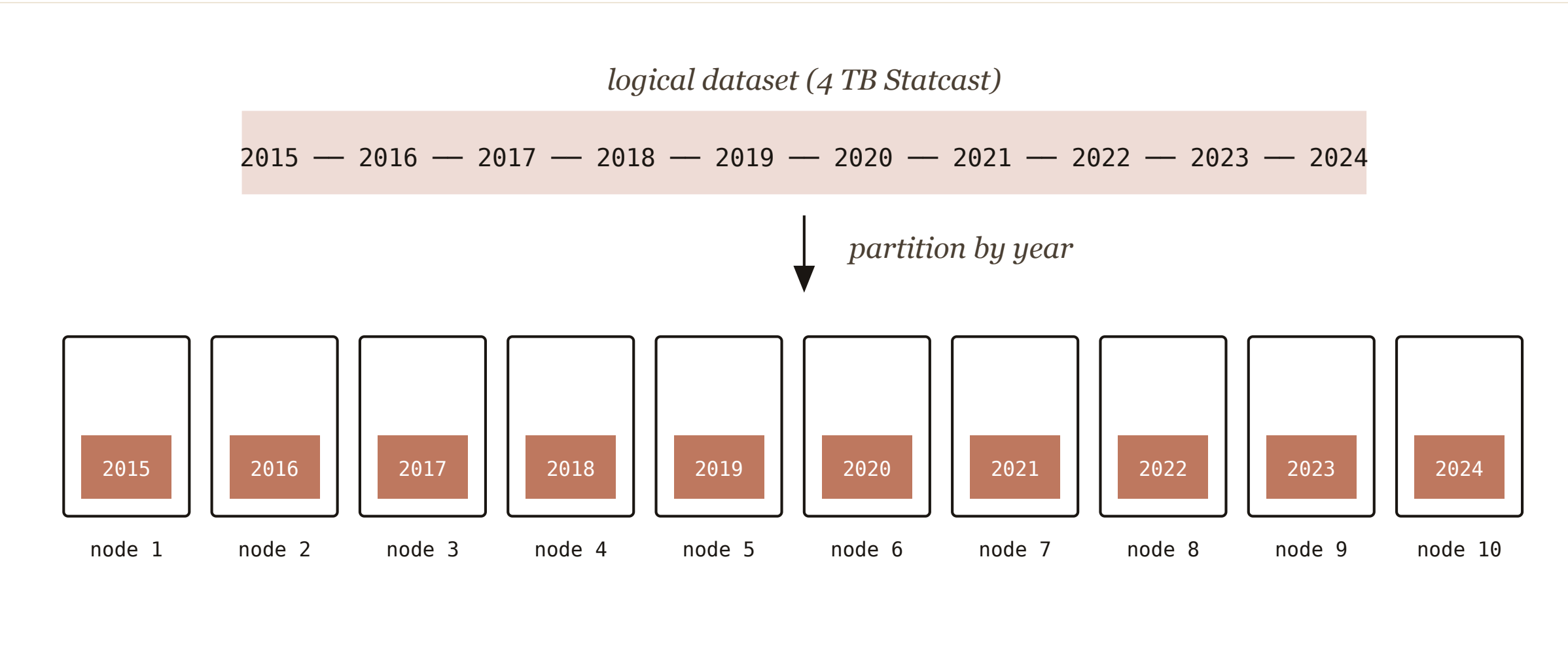

FIG 1.3 *Partitioning the Statcast dataset by year across ten nodes. Each node holds one logical partition. The dataset still exists conceptually as one whole, but no single machine holds more than 10% of it.*

Partitioning bought us something beyond just making the data fit. It also gave us parallelism. If MIDN A only needs the 2020 season, exactly one node has to do work. If MIDN B needs an analysis across all ten years, all ten nodes can work simultaneously, each on their own slice, and combine the results at the end. We have turned a 4 TB problem into ten 400 GB problems that can be solved in parallel. This is the single most important benefit of partitioning, and it is going to reappear when we get to Spark in Unit 2.

How do we decide how to partition? The choice depends on the query patterns. For Statcast we partitioned by year because most queries are scoped to a year or a range of years. If most queries asked "show me everything by pitcher X across all seasons," then partitioning by pitcher would be a better choice. The general principle: partition the data so that common queries touch as few partitions as possible. We will see this rule again, in much sharper form, when we talk about Spark's predicate pushdown and partition pruning.

So far so good. Lets ask the next question. What happens when one of the ten machines dies?

The second problem: nodes die

Node 6 has just lost its disk. From the previous class we know this is the common case, not the exception. The 2020 season has just become unreachable. The other nine seasons are fine, but if MIDN B's query needs 2020 data, the answer is now "wait, that data doesn't exist anymore." That is not an acceptable outcome.

The fix is unsurprising. Store multiple copies. If 2020 lives on node 6 *and* on node 2 *and* on node 9, then losing any one of those nodes does not lose the data. This is the principle of **replication**: storing multiple copies (called **replicas**) of the same data on multiple nodes.

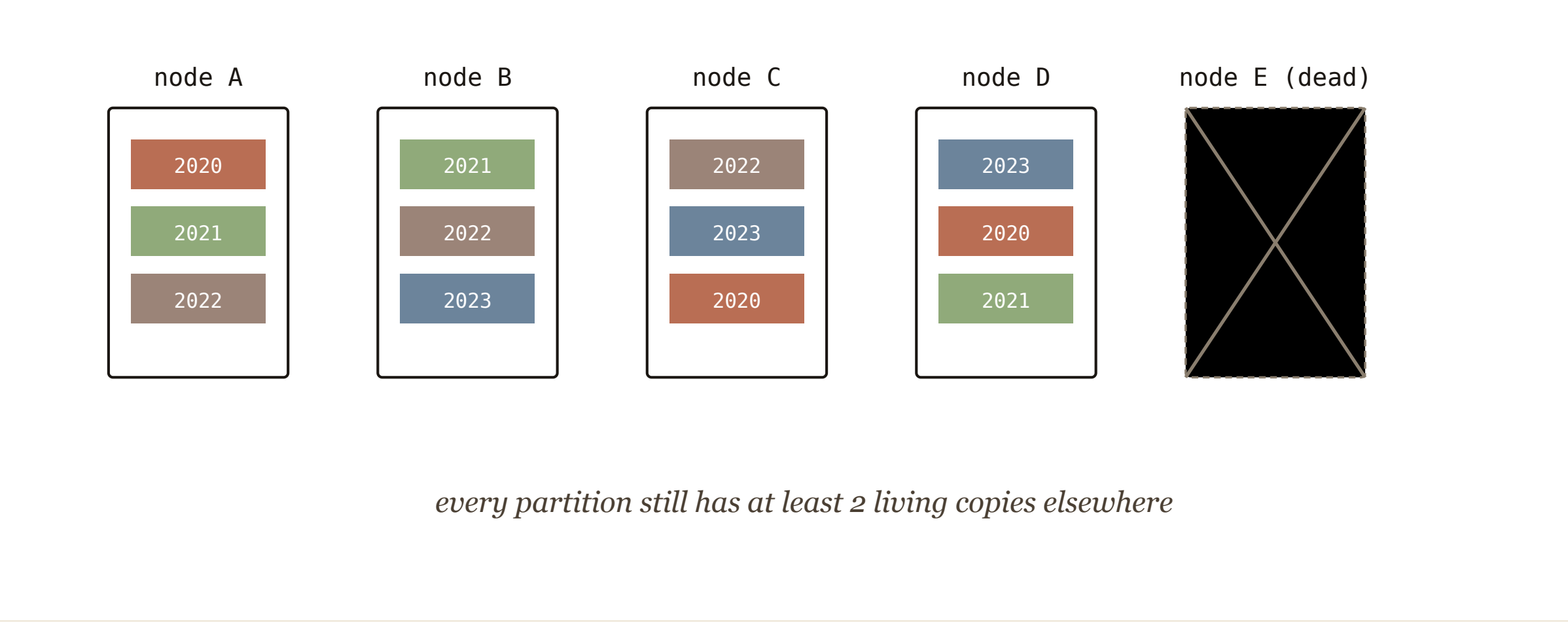

FIG 1.4 *Replication with replication factor 3. Each partition (color-coded by year) is stored on three different nodes. Even with node E down, every partition is reachable. Storage cost is 3× the raw data size; data-loss tolerance is one node failure per partition at any given moment.*

The number of copies is called the **replication factor**. Three is the canonical default for reasons we will come back to in a moment. Storing three copies means we use 3× the raw disk for 1× the logical data, which sounds expensive until you remember that disks are cheap and rewriting your data from a backup every time a disk dies is expensive.

Take a moment to think about when replication is most useful. Would a system with three replicas of each partition scale better with read-heavy or write-heavy workloads? Lets walk through it.

For reads, replication is a clear win. If MIDN A wants to read the 2020 season and three nodes have a copy of it, the system can answer the read from whichever copy is closest or least busy. Three nodes can serve three concurrent readers in parallel. Read throughput scales roughly linearly with the replication factor. Replication is good for read-intensive workloads.

For writes, replication is a tax. If MIDN B wants to update the 2020 season, all three copies have to be updated, otherwise the next read might come back with a stale answer. So a single write becomes three writes. Write throughput is hurt by the replication factor, not helped by it. Replication is a cost on write-intensive workloads.

This isn't fatal. Most analytics workloads (and Statcast definitely qualifies) are dramatically read-heavy: the data is written once when the season ends and read thousands of times for analysis. The economics work in our favor. But if your workload is write-heavy, you will need different tools, which is why we look at object storage and Kafka later in this unit and the next.

The third problem: who's in charge?

We have partitioned the data and replicated each partition. Now we have a new problem we didn't have before. With ten machines holding pieces of a logical dataset, who knows what's where? When MIDN A asks for the 2020 season, somebody has to know that the 2020 season lives on nodes A, C, and D. When node E dies, somebody has to notice and ask other nodes to make new copies of E's partitions. Somebody has to be in charge.

There are two design choices, and which one you pick shapes everything downstream.

Architecture choice 1: master-worker

The **master-worker** architecture (sometimes called leader-follower or primary-secondary; the field has been working on the names) designates one special node as the master. The master knows where every partition lives. The workers store the actual data. When a client wants to read or write, it asks the master where to go, and then talks directly to the workers.

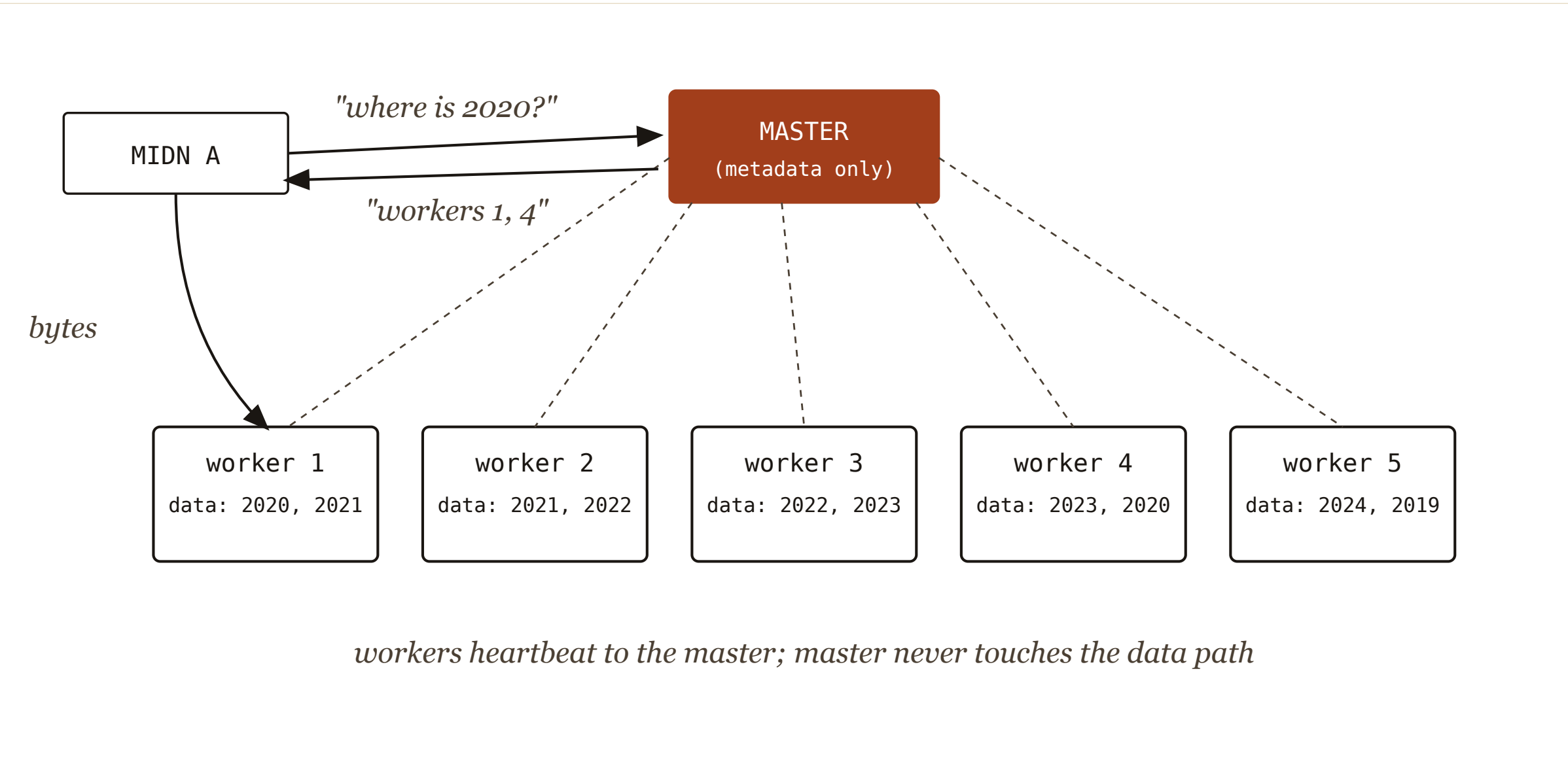

FIG 1.5 *Master-worker architecture. The master holds the map from partition to worker. Clients ask the master once, then talk to the workers directly. The master is never in the bulk-data path.*

The master-worker design is conceptually simple, and that simplicity is its biggest strength. There is one authoritative source of truth for where everything lives. Decisions about replication, recovery, and rebalancing all happen in one place. Adding a worker is straightforward: the new worker introduces itself to the master, and the master starts using it.

The weakness is also obvious. The master is a single point of failure. If the master dies, nobody knows where anything is, and the cluster is effectively down even though every byte of data is still safely on the workers. The master is also a potential bottleneck: every read and write has to go through it at least for metadata. If the cluster is large enough, the master runs out of memory to hold the metadata, or runs out of CPU to answer client requests.

The mitigation for the single-point-of-failure problem is straightforward: run a standby master that watches the live one, and promote the standby if the live master dies. The mitigation for the bottleneck problem is to keep the master out of the data path (as in Fig. 1.5) and to make sure the metadata is small relative to the data. We will see HDFS apply both mitigations.

Architecture choice 2: peer-to-peer

The other choice is to give up on having a master at all. In the **peer-to-peer** architecture, every node is equal. There is no special "in charge" node. Knowledge of who has what is replicated across all the nodes, usually via a gossip protocol where each node tells a few neighbors what it knows and those neighbors propagate the news.

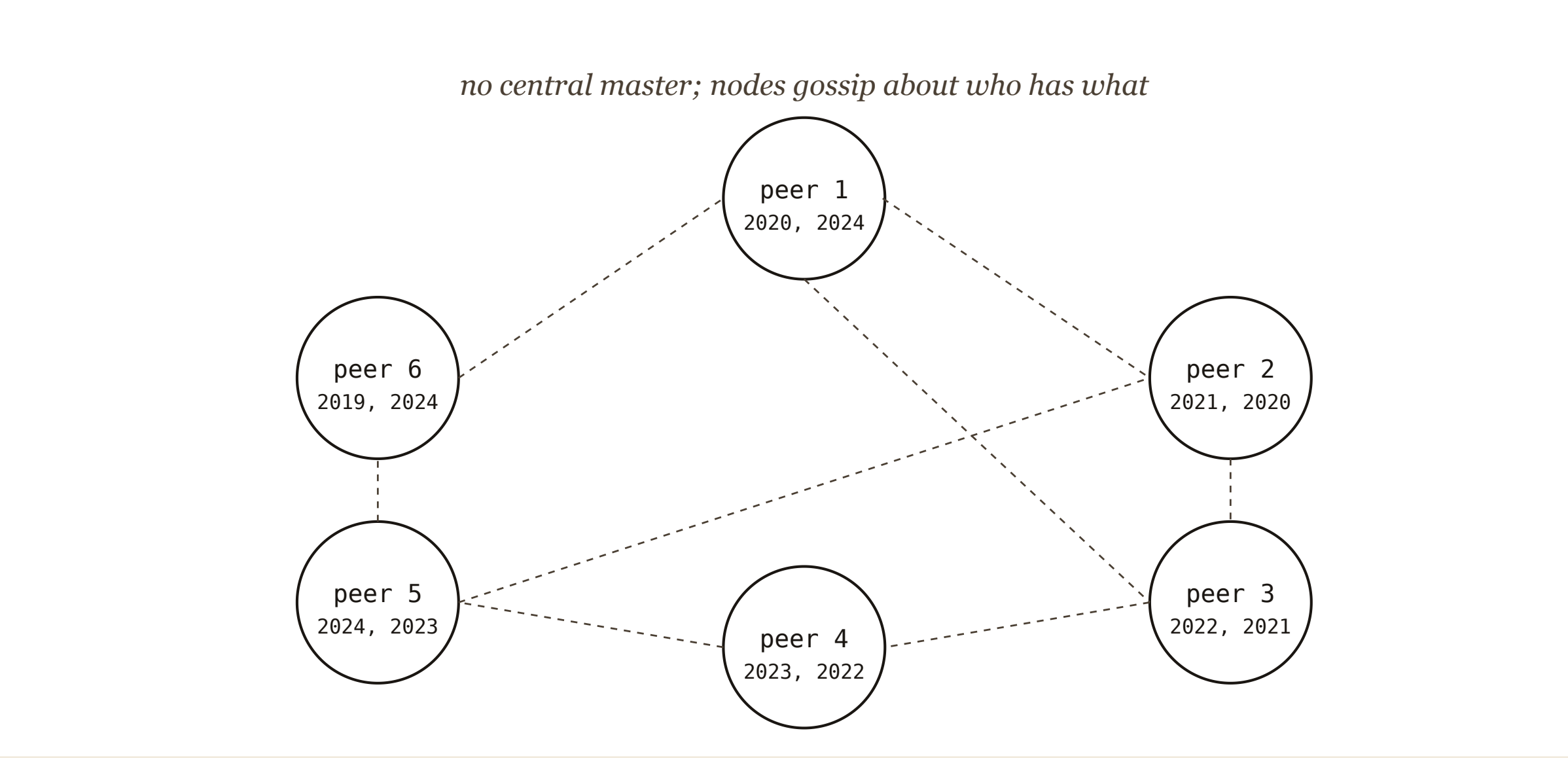

FIG 1.6 *Peer-to-peer architecture. Every node is equal. Knowledge propagates by gossip. There is no single point of failure, but there is also no single source of truth.*

The peer-to-peer design has no single point of failure. Any node can die and the cluster keeps working, because no node was special. It also scales horizontally without bound, because there is no centralized component to become a bottleneck. Cassandra and DynamoDB are notable peer-to-peer systems.

The price is complexity. Without a single source of truth, deciding which copy of the data is the "right" copy becomes hard. If MIDN B writes to peer 2 and then immediately reads from peer 3, the gossip might not have propagated yet, and peer 3 might return the old value. The whole class of problems that consistency tries to solve gets much harder to reason about. We will pick this thread next week.

HDFS: a worked master-worker example

With the two principles (partitioning, replication) and the two architectures (master-worker, peer-to-peer) in hand, lets look at HDFS, the cleanest master-worker example we have. It is no longer the cutting edge of how big-data storage is deployed (we will talk about object storage next), but it is the cleanest mental model for what a distributed file system *is*, and once you understand HDFS the alternatives are easy to read.

HDFS gives the master a job-specific name: the **NameNode**. The workers are called **DataNodes**. The partitions are called **blocks**, and the default block size is 128 MB.

That block size is worth pausing on. A local filesystem like ext4 uses 4 KB blocks, four orders of magnitude smaller. The reason HDFS uses 128 MB blocks is that the per-block overhead in the NameNode (one record in memory for each block) is roughly constant regardless of block size, and the cost of seeking on a spinning disk is roughly constant regardless of block size. So you want each block to be large enough that the metadata is small relative to the data, and large enough that the seek cost is small relative to the read cost. 128 MB is a number that lands in the right neighborhood for both.

Now lets trace what happens end-to-end.

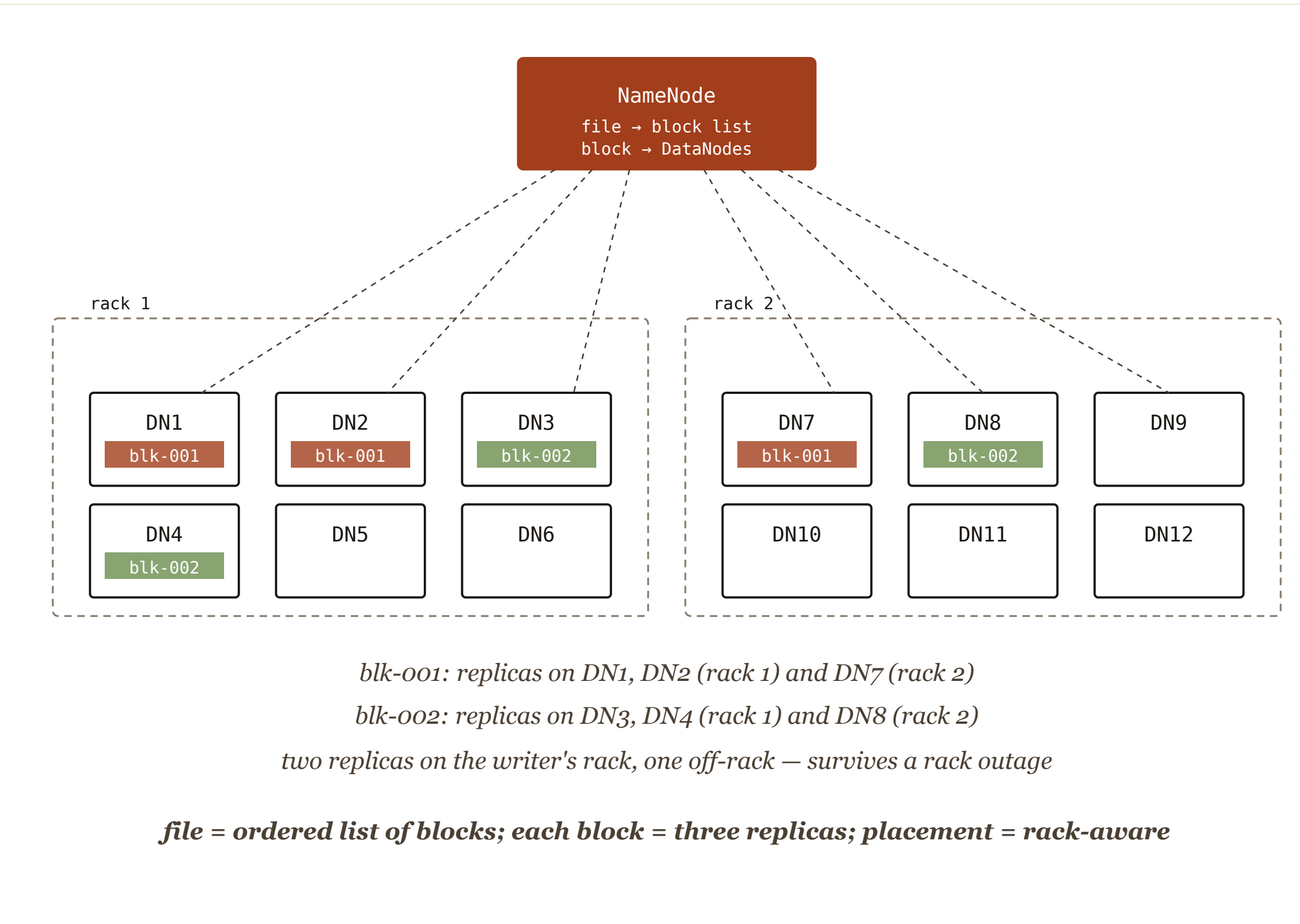
NameNode
file → block list
block → DataNodes
rack 1
rack 2
DN1
blk-001
DN2
blk-001
DN3
blk-002
DN7
blk-001
DN8
blk-002
DN9
DN4
blk-002
DN5
DN6
DN10
DN11
DN12
blk-001: replicas on DN1, DN2 (rack 1) and DN7 (rack 2)
blk-002: replicas on DN3, DN4 (rack 1) and DN8 (rack 2)
two replicas on the writer's rack, one off-rack — survives a rack outage
file = ordered list of blocks; each block = three replicas; placement = rack-aware

FIG 1.7 *HDFS with two racks and twelve DataNodes. Each block is replicated three times. Two replicas stay on the writer's rack to keep most reads cheap; the third goes off-rack so a rack-level outage doesn't lose data. The NameNode holds only metadata and never moves bulk bytes.*

Two new ideas in this figure beyond what we have already developed.

First, **rack awareness**. The three replicas of a block are not placed randomly. HDFS knows which DataNode is in which rack (this is configured at cluster setup) and uses that knowledge with a specific policy:

- Replica 1: on the rack of whoever wrote the block.
- Replica 2: on a different rack.
- Replica 3: on the same other rack as Replica 2.

Take a moment to think about why this is the policy. What kinds of failure does it tolerate, and what does it cost? If a single DataNode fails, two replicas remain. Fine. If an entire rack loses power (a real thing that happens, because racks share power supplies and top-of-rack switches), we still have at least one replica on the other rack. Fine. We have tolerated rack-level failure, not just node-level failure. The cost is that one of our three writes has to cross the rack boundary, which is the slow part of the datacenter network. We pay one expensive cross-rack write to get rack-level fault tolerance. That is a deliberate tradeoff, and it is the kind of tradeoff this course will keep asking you to recognize.

Second, **heartbeats and block reports**. Every DataNode sends a short message to the NameNode every few seconds saying "I'm still alive." Every few minutes it sends a longer message listing every block it holds. The NameNode uses the heartbeats to detect dead DataNodes (no heartbeat for ten minutes means the node is presumed dead) and uses the block reports to maintain its file-to-block-to-DataNode map. When the NameNode notices a DataNode has died, it consults its records, sees which blocks lived there, and instructs other DataNodes to make new copies until every block is back to three replicas. The cluster heals itself. No human is paged. This is the operational regime we wanted.

The write path, end to end

When MIDN B writes a new 500 MB file to HDFS, here is what happens:

1. The client asks the NameNode: I want to write `/data/foo.parquet`. The NameNode allocates the first block's ID, picks three DataNodes for the replicas (rack-aware), and replies with the list.
2. The client streams the bytes to the first DataNode in the list. That DataNode is *simultaneously* streaming those bytes onward to the second DataNode, which is streaming them onward to the third. This is called a **pipeline write**.
3. When the first block (128 MB) is fully acknowledged by all three replicas, the client asks the NameNode for the next block's allocation. Repeat until the 500 MB file is done. A 500 MB file is four blocks: three full 128 MB blocks plus one 116 MB tail.
4. The client tells the NameNode "I'm done." The NameNode commits the file to its persistent metadata. The file is now visible to other readers.

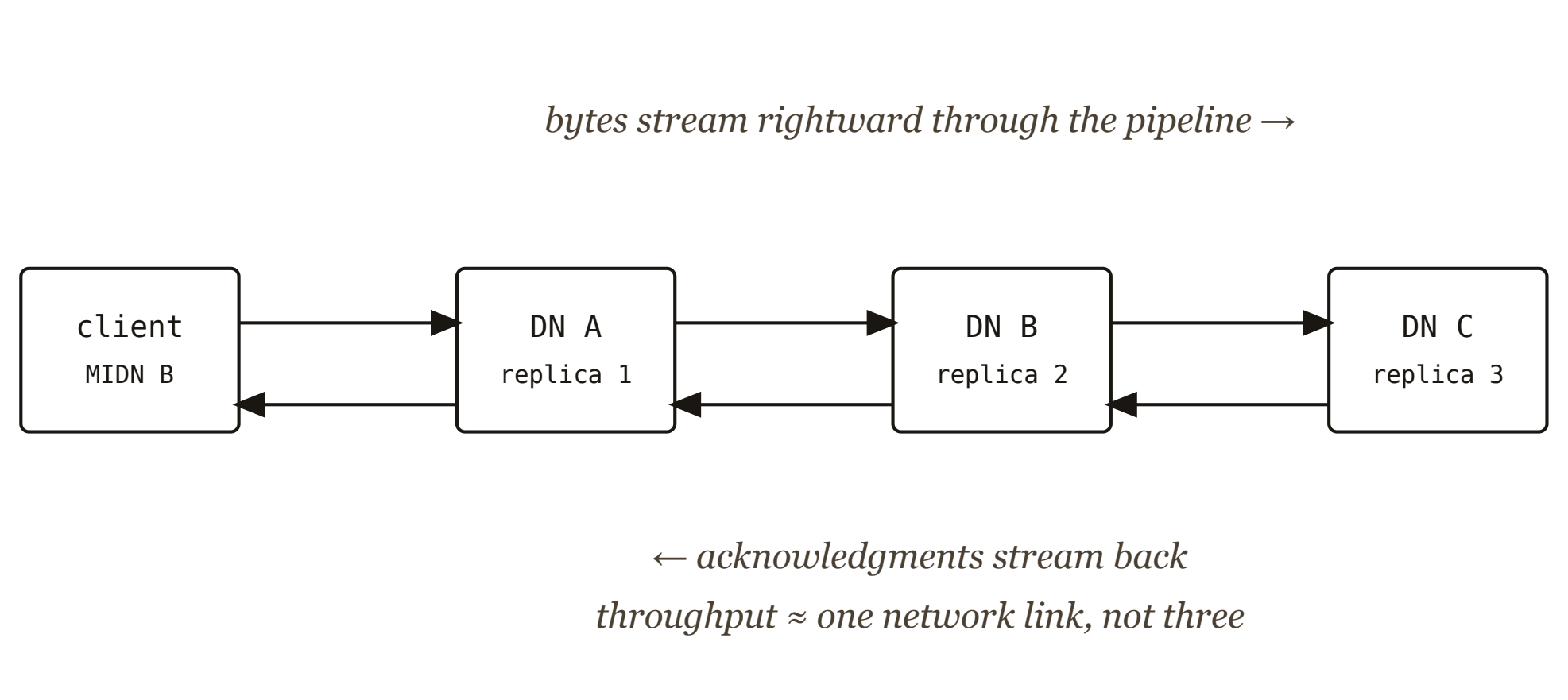
bytes stream rightward through the pipeline →
client
MIDN B
DN A
replica 1
DN B
replica 2
DN C
replica 3
← acknowledgments stream back
throughput ≈ one network link, not three

FIG 1.8 *The HDFS pipeline write. Bytes flow client → DN A → DN B → DN C in a single chain, with each DataNode forwarding to the next as it receives. Throughput is gated by one network link, not three. If you wrote to the three replicas sequentially you would be three times slower.*

The pipeline write is important because it means write throughput is gated by one network link, not three. If the client wrote to each replica sequentially (write to DN A, wait, then write to DN B, wait, then write to DN C, wait), the write would be three times slower. The pipeline cuts that cost down to one link's worth of bandwidth, which is the best you can do given the physics.

Where HDFS hurts

HDFS is a good first design. It is not the design most modern systems use, and the reasons are worth knowing now because they motivate the next class.

1. **One NameNode.** All metadata fits in the NameNode's memory. Each file and each block consumes around 150 bytes of NameNode heap. A cluster with a billion files and three billion blocks needs hundreds of GB of RAM just to hold the metadata. That's not impossible but it is expensive, and it caps the maximum cluster size.
2. **Compute and storage are bolted together.** A DataNode is a machine with disks *and* a CPU. If you need more compute, you have to add storage with it. If you need more storage, you get compute with it. The two scale together whether you wanted that or not.
3. **Storing cold data is expensive.** Even data nobody has read in two years sits on the cluster paying the full three-copy replication tax on expensive datacenter disks.

These three pains motivate object storage, which is what we pick up next.

Object storage and the modern *lakehouse*

We have a working mental model for HDFS. Lets ask the question every junior data scientist eventually asks: what would you change about HDFS if you could redesign it from scratch today? Three things show up immediately.

1. Decouple metadata from a single server.
2. Decouple compute from storage.
3. Make cold data cheap.

That is, essentially, **object storage**. AWS S3 is the canonical example, having launched in 2006 and pioneered the design. We will use **MinIO** in the lab, which speaks the same S3 API and behaves the same way for our purposes but runs in a Docker container on your VM. The interface is identical; only the implementation underneath differs.

What an object store actually is

An object store is a flat key-value store. The key is a string, something like `statcast/year=2024/month=04/day=07/pitches.parquet`. The value is a blob of bytes that could be one byte or 5 TB. There is no directory tree. There is no inode. There is no notion of "this file lives on this server." The store gives you four operations: PUT, GET, DELETE, LIST. That is it.

Notice what is missing. There is no `seek()` within an object. There is no `append()` to an object. There is no `rename()`. An object is immutable: once written, the only ways to change it are to overwrite it entirely or delete it. If that sounds restrictive, hold the thought. It is the same restriction that, in exchange, buys us almost all of the operational properties we wanted.

Two things to note about the "directory" structure. First, the slashes in the key are not real directories. The key is a single string; the slashes are just characters. There is no `statcast/` directory anywhere. But because LIST supports prefix filters, you can ask "give me all keys that start with `statcast/year=2024/`" and it behaves *as if* there were a directory there. This naming convention is so universal that we treat it as the directory structure even though it isn't really one.

Second, the `year=2024/month=04/day=07/` pattern is called **Hive-style partitioning** and is how every modern data tool expects directories to be named. Spark, Presto, DuckDB, and dozens of others know to look for `column=value` patterns in the path and use them for partition pruning. We will exploit this directly in Unit 2.

How is it implemented?

You should not, today, need to know the internals of S3 to use it. The interface is well-defined and the implementation is somebody else's problem. But the mental model matters because it will explain the performance and cost surprises later. The internals look roughly like this.

The data is sharded across many storage nodes and replicated or erasure-coded (which is replication's more storage-efficient cousin; instead of storing 3 copies of each block, erasure coding stores enough redundant parity that you can reconstruct any block from a subset of its parts). The metadata service that maps keys to physical locations is itself distributed, so there is no single-NameNode bottleneck. The whole thing sits behind an HTTP API. You don't talk to the storage nodes directly; you talk to a load balancer, which routes your request.

Compare this to HDFS. The "one NameNode" problem is gone, because the metadata service is itself horizontally scaled. The "compute and storage are bolted together" problem is gone, because your Spark cluster has zero disks of its own; it reads from S3 over HTTP. The "cold data is expensive" problem has been turned into a product feature: S3 lets you move objects to colder storage classes (Standard-IA, Glacier, Glacier Deep Archive) that cost a fraction of the hot-tier rate, at the price of longer retrieval times.

HDFS vs S3: the tradeoff sheet

Object storage is not strictly better. It is differently good. Lets walk through what changes.

Dimension	HDFS	S3 (object storage)
Throughput	Local disk reads; very fast when compute is co-located with data	Over HTTP; slower per-byte but parallelizes across many connections
Latency	Microseconds per request	50–100 ms first-byte latency per request
Listing	Fast directory operations	LIST is paginated and slow; the worst gotcha in practice
Consistency	Strong (single NameNode)	Strong since 2020 (was eventual before; old docs are out of date)
Per-GB cost	Expensive: full server hardware	Cheap: ~$23/TB-month for hot, far less for cold tiers
Per-request cost	None	Real: ~$5 per million PUT, ~$0.40 per million GET
Scaling	NameNode memory caps cluster size	Effectively unbounded
Operational burden	You run the cluster	AWS runs the cluster

A few of these deserve more attention than the table can give them.

Throughput. HDFS, when your compute is on the same nodes as the data, lets you read from local disk. S3, even on the fastest networks, makes you go over HTTP. The local HDFS read is the faster path in raw bytes-per-second for a single reader. But HDFS only wins when your compute is co-located with the data, which is exactly the thing object storage gave up on purpose. With S3, you can throw twenty compute nodes at one bucket and read in parallel from twenty connections, and the aggregate throughput often exceeds what HDFS can deliver. The architecture changed, not just the storage.

Latency. First-byte latency in S3 is 50 to 100 milliseconds. That is a real cost. For workloads that read a single small file, this matters. For workloads that read large files sequentially (every workload in this course), it does not, because the per-request cost gets amortized across the bytes. The lesson: object storage rewards big files and punishes small ones, which is going to be a theme.

Listing. Asking S3 "what objects exist under this prefix" is slow. The LIST operation is paginated (1,000 results per call), runs at the rate of the metadata service, and gets linearly slower as the number of objects under the prefix grows. If your job does `ls` a lot (and many big-data jobs do, every time they read a directory), you will feel this. This is one of the things the lakehouse metadata layer fixes by keeping its own index.

Consistency. S3 is now strongly consistent for object reads and writes, as of December 2020. Before that, S3 had a complicated mix of strong consistency for some operations and eventual consistency for others, and big-data tools had to work around it. If you read old big-data papers or old StackOverflow answers about "S3 eventual consistency," that warning is out of date. PUT-after-PUT, PUT-then-GET, and LIST-after-PUT are all strongly consistent now.

Cost. Object storage is dramatically cheaper per GB than HDFS at scale, especially when you factor in the operational burden. The catch is the per-request and per-byte-transferred charges. If your job does many small requests, you can run up a real bill. If your job downloads the data out of AWS to your VM, the egress charges are even more real. Lab 2 will let you feel some of this directly with MinIO, which doesn't actually bill you, but which lets you observe the request patterns.

The modern lakehouse story

Once we say "the data lives in object storage and the compute is separate," a new pattern emerges. We can have:

- A bucket of Parquet files in S3 holding the data. Cheap, durable, accessible.
- A Spark cluster, spun up on demand, that reads from S3, does the work, and writes results back to S3. Compute is paid for only while running.
- A metadata layer on top of the Parquet files that gives us things object storage took away: efficient listing, atomic appends, schema evolution, time travel. This is what Delta Lake, Apache Iceberg, and Apache Hudi all do. They differ in details; for the purposes of this course, treat them as variations on the same idea.

The architecture has a name: the **lakehouse**. Data lake economics (cheap object storage, open formats) with warehouse semantics (transactions, schema, point-in-time queries). Almost every new analytics deployment in the past five years is some flavor of this. If you read a job posting for a data engineer in 2026, "experience with Delta Lake, Iceberg, or Hudi" is on most of them.

The lakehouse metadata layer is conceptually a small NameNode that lives *alongside* your data files. It holds the list of which Parquet files belong to which logical table, what each file's schema is, what statistics are available about each file, and a log of every change. Reads consult the metadata first to figure out which files to actually open. Writes append new files and append a new entry to the log. There is no rewrite-in-place; immutability is preserved. We will use **Delta Lake** in Unit 2 as our representative.

Where this leaves us

We have decoupled compute and storage, made cold data cheap, and removed the single-NameNode bottleneck. But we created a new problem along the way: the data is now somewhere distant, and writes from two different jobs might step on each other in ways HDFS's master could detect and we no longer can. The metadata layer is what disarms that. Sounds great, right? It does for now. But the moment we say "writes from two different jobs," we have walked back into the consistency conversation.

Consistency, *CAP*, and PACELC

We have set up the cluster shape and the storage layer. Now we have to talk about what happens when multiple clients read and write the same data. This is the conversation that most data scientists wish they had absorbed earlier. The reason it matters: every distributed system you encounter, including ones the marketing materials describe as "fully consistent" or "always available," is making specific tradeoffs against this material, and being able to read those tradeoffs is most of what makes you a useful data scientist.

A recap of consistency from SD321

SD321 introduced ACID transactions in the context of relational databases: Atomicity, Consistency, Isolation, Durability. The "C" in ACID is about preserving database invariants (foreign keys, check constraints, things like "this account balance can't go negative"). When we say "consistency" in distributed systems, we mean something different and a little more fundamental. Distributed-systems consistency is about: *after a write completes, what do subsequent reads see?*

The strongest answer: every read sees the most recent write. That's called **strong consistency**. It is what every single-machine database gives you by default and what most programmers assume by reflex.

Weaker answers: a read might see a stale value for some bounded time, or might see a stale value forever until you read from a specific replica, or might see values that arrive out of the order they were written. Each of these is a real point on the spectrum and each has been someone's design choice in a real system.

The network partition problem

Take the master-worker setup from class 2. Two workers. The master holds the data on both workers (replicated). MIDN A writes `balance = $100` through the master. The master updates worker 1, and is in the middle of updating worker 2 when something happens: the network link between the master and worker 2 goes down.

A **partition** in this context (different from the partitioning we did for storage, frustrating overlap of terminology) is a situation where the network between some subset of your nodes and the rest is broken. Maybe a switch died. Maybe a fiber got cut. Maybe just a bad cable. The two halves of the cluster can each still talk to themselves, they just can't talk to each other. Each half might not even know the other exists right now; from its perspective, the other half just stopped answering.

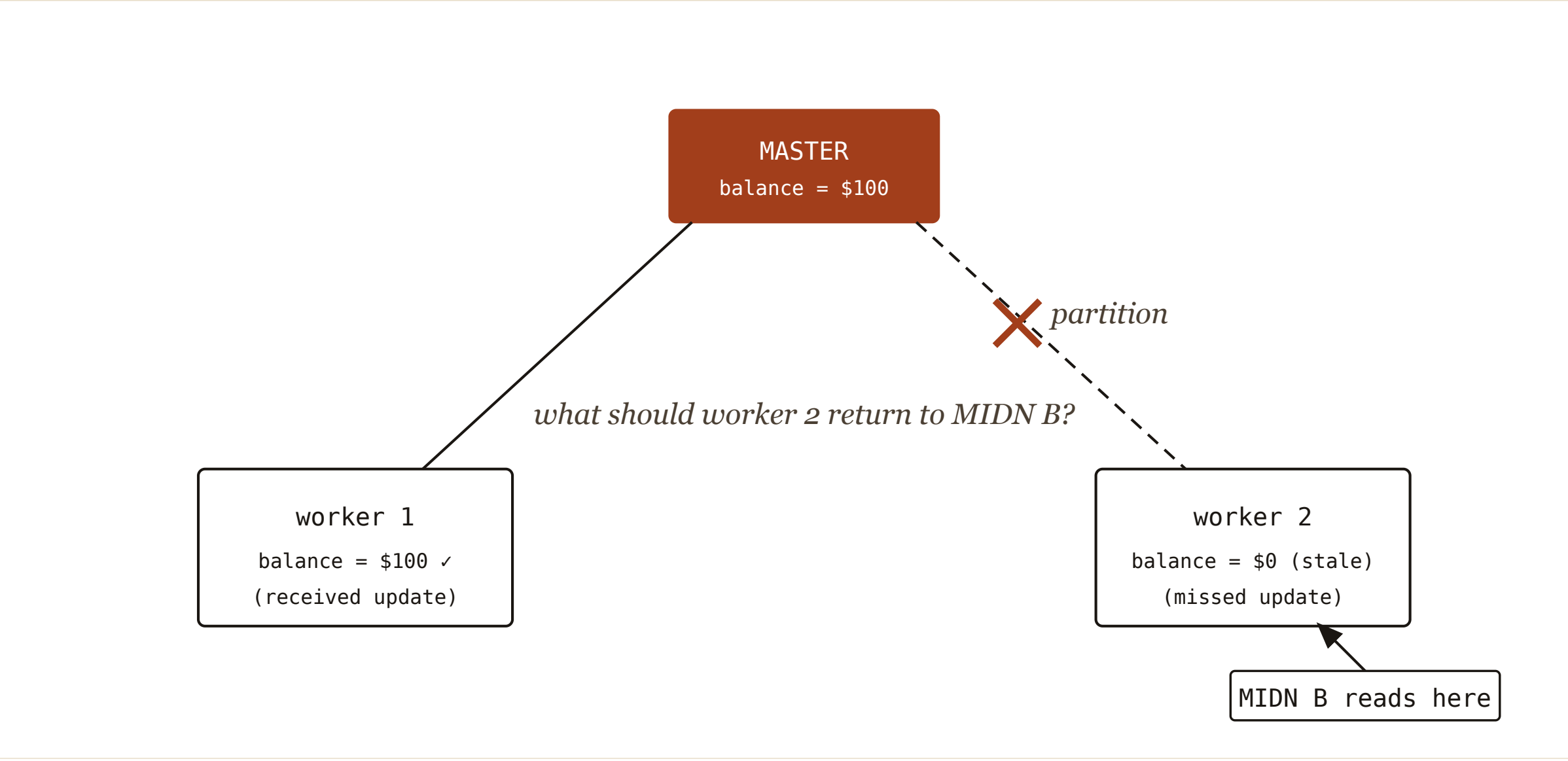

FIG 1.9 *A network partition during replication. MIDN A's write reached worker 1 but not worker 2. MIDN B is reading from worker 2. There are exactly two honest answers worker 2 can give, and they trade off in different directions.*

MIDN B connects to worker 2 (maybe because worker 2 is closest, or because of load balancing) and asks for the balance. What should worker 2 return?

There are exactly two honest options. Option one, worker 2 returns the old value, $0. The system is *available* (MIDN B got a response), but *inconsistent* (the response is wrong, because the actual current balance is $100). Option two, worker 2 refuses to answer until it can confirm with the master that it has the latest data. The system is *consistent* (no wrong answers go out), but *not available* (MIDN B is hanging, waiting for a confirmation that may never come).

You do not get to pick "both." There is no third clever answer. The network can drop your messages, and from inside the system you cannot distinguish between "the master is slow" and "the master is gone." If you insist on consistency, you must refuse to answer until you've confirmed; if you insist on availability, you must answer with what you have. It is the heart of the CAP theorem.

The CAP theorem

The CAP theorem, originally conjectured by Eric Brewer in 2000 and proven by Gilbert and Lynch in 2002, says: a distributed system can guarantee at most two of:

- **Consistency**: every read sees the most recent write (strong consistency in the sense above).
- **Availability**: every request gets a response (success or failure, not a hang).
- **Partition tolerance**: the system keeps working even when the network drops messages between nodes.

"Choose 2 of 3" is a lie

Here is where most CAP explanations go wrong. They say "pick any two." That implies you could pick CA: consistent and available, no partition tolerance. You cannot pick that. If your system runs on a real network (and every distributed system runs on a real network), partitions will eventually happen. Refusing to tolerate them does not mean they won't happen; it means your system breaks when they do.

So the honest framing of CAP is: P is non-negotiable in any real distributed system. The choice is between **CP** (sacrifice availability during partitions, return errors or hangs rather than wrong data) and **AP** (sacrifice consistency during partitions, return possibly-stale data rather than errors). Two real choices, not three.

CP systems include ZooKeeper, etcd, HBase, MongoDB (in its strict configuration), and Google Spanner. These refuse to serve requests they aren't sure about. They are appropriate when wrong data is worse than no data: configuration stores, financial ledgers, distributed locks, leader-election services.

AP systems include Cassandra, DynamoDB (in its default configuration), Riak, CouchDB, and most "eventually consistent" key-value stores. They serve possibly-stale data during a partition. They are appropriate when no data is worse than slightly-stale data: shopping carts, social feeds, view counters, sensor telemetry.

Note that "CP" doesn't mean "the system has zero availability" and "AP" doesn't mean "the system has zero consistency." Both terms describe what happens *during a partition*. Outside of partitions, both can be plenty consistent and plenty available. The categorization is about the choice the system makes when forced to choose.

PACELC: what about when there is no partition?

CAP gives you the choice during a partition. What about the 99.99% of the time when there isn't one? CAP is silent on that, which is unsatisfying because that's most of operational reality.

PACELC, formalized by Daniel Abadi in 2010, extends CAP to address this:

The PACELC formulation

If there is a **P**artition, choose between **A**vailability and **C**onsistency (PA / PC). **E**lse, choose between **L**atency and **C**onsistency (EL / EC).

The new part is the "else" clause. Even with no partition, if you want consistency, you have to pay for it. Specifically, you have to coordinate between replicas before answering a read, which costs you a network round trip. Remember Dean's numbers from the first day of class — roughly 500 microseconds within a datacenter, much worse across regions. If you want low latency, you have to be willing to answer from a single replica without coordinating, which means you might be wrong by a few milliseconds' worth of writes.

A system is described by two letters: its partition-time choice (PA or PC) and its no-partition choice (EL or EC).

System	PACELC	Notes
Cassandra	PA/EL	Available during partitions, low-latency reads otherwise; consistency level is tunable per-query.
DynamoDB (default)	PA/EL	Eventually consistent reads by default; can opt into strong with extra latency and cost.
HBase	PC/EC	Region servers are single-master per region; values prioritize correctness over availability.
MongoDB (default)	PA/EC	Default reads from the primary (consistent) but failover during a partition can serve stale data.
Google Spanner	PC/EC	Uses atomic clocks (TrueTime) to maintain strong consistency at global scale; pays the latency.
S3	PA/EC	Strongly consistent for object reads/writes; available during typical network issues.

The reason PACELC matters more than CAP in practice is that you spend most of your operational life in the "else" clause. Partitions are rare. The latency-versus-consistency tradeoff is happening on every single request. When you choose a database, you are mostly choosing its EL-or-EC posture, not its PA-or-PC posture.

What this means for the rest of this class

For the rest of this course, when we talk about a storage or streaming system, we will name its PACELC position. HDFS is PC/EC, with the NameNode as the coordinator. S3 is PA/EC for individual objects. Kafka is configurable: you can run it as PC/EC with full replication acknowledgement, or PA/EL with relaxed acknowledgement; we will turn those knobs ourselves in unit 3. Spark itself does not store data, but its checkpointing and write semantics rest on whatever storage layer you put underneath, so the PACELC of the storage propagates into the PACELC of the job.

The mental discipline this gives you is small but real: when you see a new system, ask its PACELC. The answer tells you what kinds of workloads it's appropriate for without your having to read the whole manual.

Row vs columnar, and what *Parquet* actually does

Up to now we have talked about *where* the data goes (which nodes, which racks, which replicas, which buckets). Now lets talk about *how the bytes are arranged* once they get there. The file format is, in our experience, the single biggest performance lever in a big-data system that students underrate. Choosing the right format on a billion-row dataset is often a 20× speedup over choosing the wrong one. Lab 3 will have you measure exactly this on Statcast.

A familiar starting point: CSV

You have been using CSV since SD211. It is the path of least resistance for data interchange. Lets be honest about what it is, though.

A CSV file is a stream of rows. Each row is a string. Each row contains all of its columns, in order, separated by commas. To find the 10,000th row, you have to read everything before it, because there is no index. To find the value of column 7 in row 10,000, you first find row 10,000 and then count commas. There is no schema; everything is text until your code parses it.

CSV is simple, and that simplicity is also its problem. Three things go wrong at scale.

First, **no types**. Every value is a string until parsed. Reading 10 million rows of CSV means parsing 10 million strings to integers, floats, dates. Parsing dominates CPU time. For wide tables (Statcast has ~80 columns), most of the wall-clock time of a "read CSV" operation is the string-to-number conversions, not the disk I/O.

Second, **no column-aware compression**. The whole row goes through the compressor as one blob. The fact that the `pitch_type` column has only 15 distinct string values across the whole file is not exploited; the compressor just sees "FF,93.2,SL,87.1,FF,94.0" mixed together and does its best.

Third, **no partial reads**. If your query is `SELECT release_speed FROM pitches`, you still have to read every byte of every row to get that one column out. You pay I/O for all 80 columns to use 1. This is the killer for analytics. Almost every analytic query reads a small subset of the columns. CSV makes you pay for all of them.

The columnar idea

What if we stored the data column-by-column instead of row-by-row? Take the Statcast schema with its 80 columns. In row layout, the bytes are arranged like this:

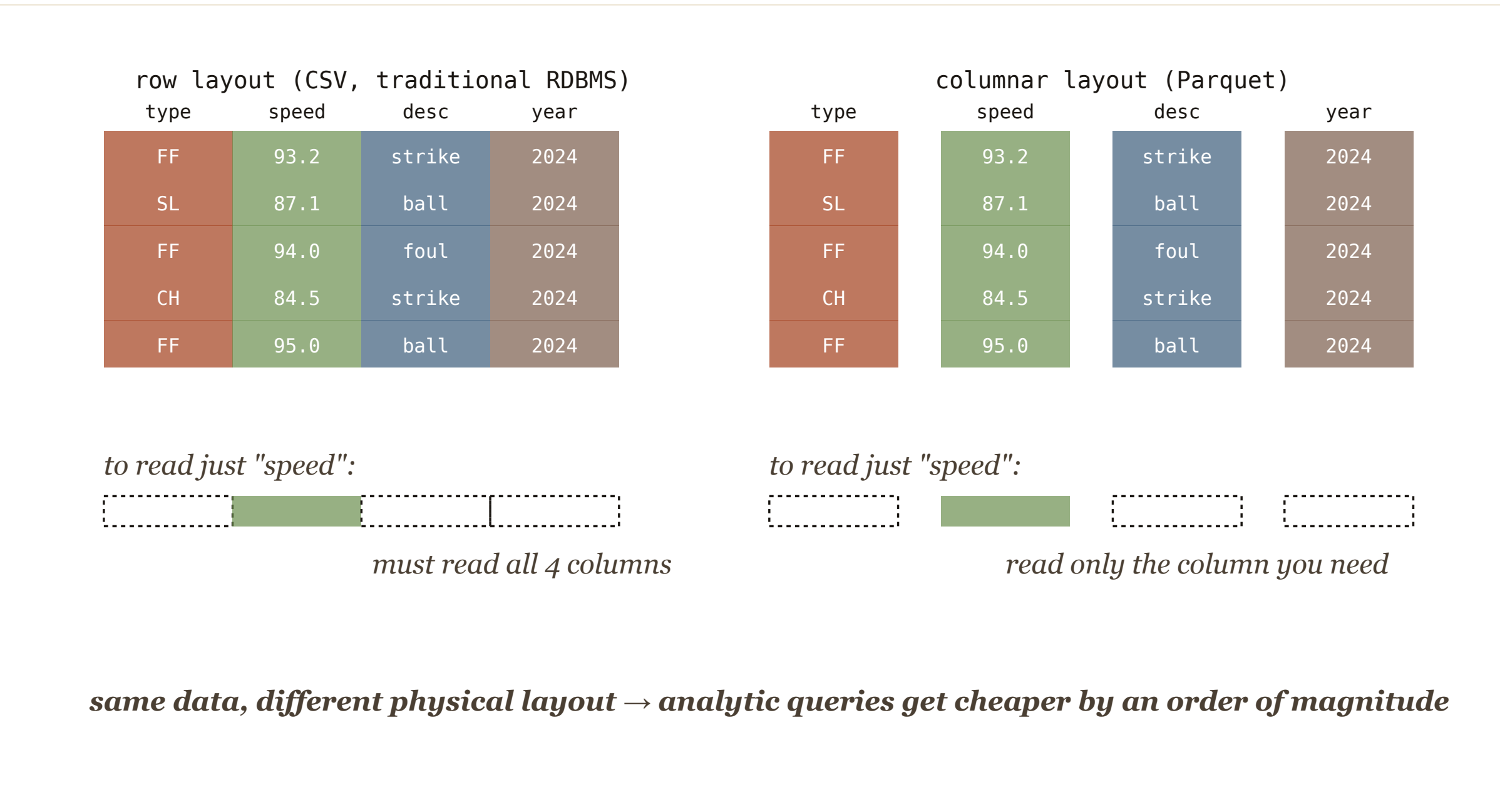

FIG 1.10 *Row vs columnar physical layout. The logical table is identical. In row layout, reading one column requires touching every other column's bytes. In columnar layout, you read only what you need.*

Same logical data, completely different physical layout. What does this buy us?

Selective column reads. `SELECT release_speed FROM pitches` now reads only the `release_speed` column. You skip the other 79 columns entirely. On Statcast that is an 80× reduction in I/O for single-column queries. Even multi-column queries usually touch a small fraction of the schema, and you pay only for what you read.

Compression that actually works. A column of `pitch_type` values has only about 15 distinct possible strings (FF, SL, CH, CU, FC, FT, KC, ...) repeated millions of times. Compressed, that column is tiny. A column of `release_speed` values is a stream of floats clustered between 65 and 105, which also compresses well. Compressing column-by-column lets each column use its own optimal codec.

Type-aware encoding. Because the format knows that the whole column is integers (or floats, or strings, or booleans), it can use type-specific tricks like dictionary encoding, run-length encoding, and bit-packing. CSV cannot do any of this because it doesn't know what the data is.

Predicate pushdown. If the format keeps per-column statistics (min, max, count), then a query like `WHERE release_speed > 100` can skip entire chunks of the file whose `release_speed` max is below 100. We will see Spark exploit this heavily in unit 2.

Parquet, specifically

Apache Parquet is the columnar format the industry has converged on. It is open, language-neutral, supported by every serious analytics tool, and is what we use for almost everything in this course. Lets walk through its internal structure, because once you can read a Parquet file's metadata, a lot of the performance behavior in Spark stops being mysterious.

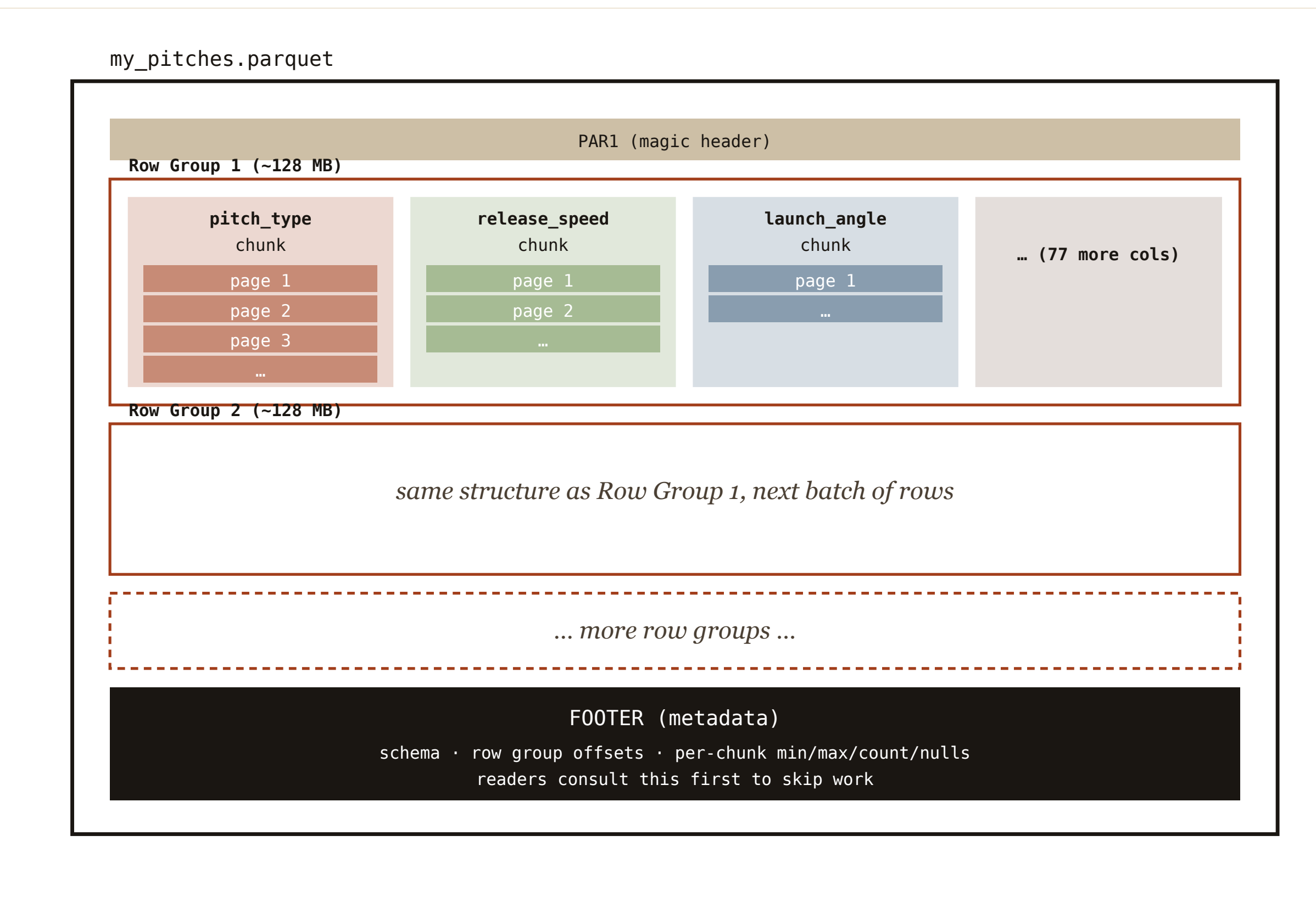

FIG 1.11 *Parquet file structure. A file is a sequence of row groups. Each row group holds column chunks (one per column). Each chunk is split into pages. The footer at the end of the file holds metadata and statistics that let readers skip whole row groups without reading them.*

A Parquet file has three levels of nesting.

Row groups. The file is split into chunks of rows called **row groups**. A typical row group is somewhere between 64 MB and 512 MB of source data; the default in most writers is 128 MB. The row group is the unit of parallelism: a Spark task processes one row group at a time. So a file with 8 row groups can be read in parallel by 8 tasks, while a file with 1 row group is processed by exactly 1 task no matter how many cores you have. (This is going to come back when we talk about partitioning in unit 2.)

Column chunks. Inside a row group, the data is laid out column by column. For each column, all of its values for that row group are stored together as a **column chunk**. Each chunk has its own encoding and its own compression codec, and each chunk records its own statistics.

Pages. Inside a column chunk, the data is further broken into smaller units called **pages**, typically about 1 MB each. Each page can be read and decompressed independently. The page is the unit of compression and the unit of read atomicity: even if you only need one value, you have to read at least one whole page.

And then, importantly, there is the **footer**: a block of metadata at the end of the file that describes everything. For each row group and each column chunk, the footer holds the min, max, count, and null count of that column's values, plus pointers to where the chunk actually lives in the file. Parquet readers consult the footer first and use it to skip work.

That last sentence is doing a lot of work, so lets show what it means.

Predicate pushdown, mechanically

You run the query `SELECT * FROM pitches WHERE release_speed > 100` against a 10 GB Parquet file. Here is what happens.

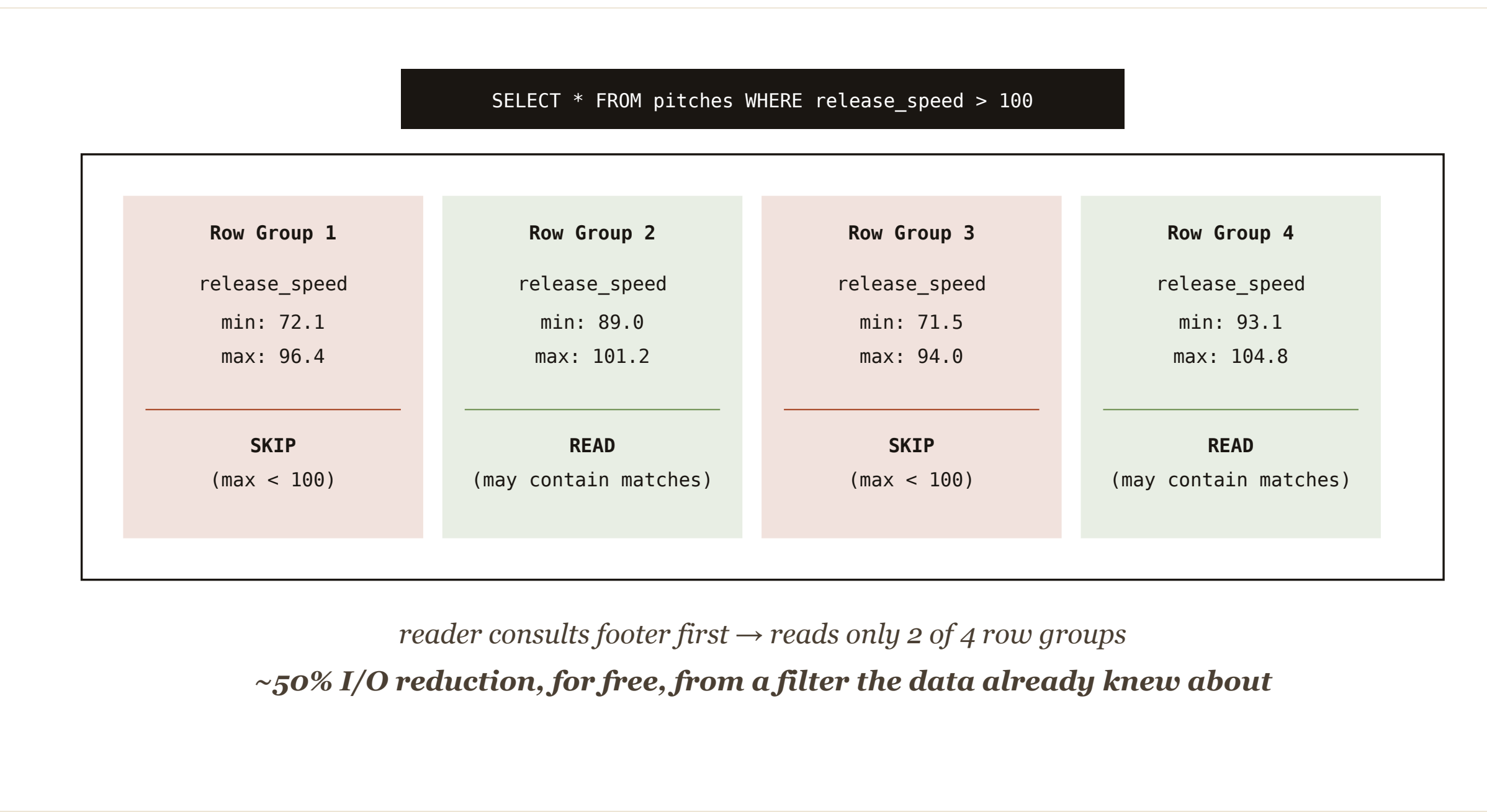

FIG 1.12 *Predicate pushdown in action. The Parquet reader checks each row group's footer statistics for `release_speed`. Row groups whose max is below 100 cannot contain any matching rows, so they are skipped entirely. The reader only opens the row groups that might contain matches.*

The reader opens the footer (a single small read at the end of the file). It looks at the statistics for each row group's `release_speed` column chunk. If the max of `release_speed` in a row group is less than 100, no row in that row group can possibly satisfy `release_speed > 100`, so the reader skips the entire row group without ever reading its bytes. This is called **predicate pushdown**: the filter from the query is "pushed down" into the storage layer, which prunes work it can prove is unnecessary.

On Statcast, where pitches cluster around 90 mph and only a small fraction are over 100, predicate pushdown often eliminates 80–90% of the I/O for the over-100 query. This happens transparently. You don't write any special code; the Parquet reader and Spark do it automatically because the format supports it.

Dictionary encoding, walked through

Lets walk through one of Parquet's more important encoding tricks, because it explains a lot of size behavior that otherwise seems mysterious.

The `pitch_type` column has roughly 15 distinct values across all of MLB: `FF` (four-seam fastball), `SL` (slider), `CH` (changeup), `CU` (curveball), `FC` (cutter), `FT` (two-seam fastball), `KC` (knuckle curve), and so on. Storing the string "FF" once per row, for 7 million rows, is wasteful: each value is 2 to 3 bytes, so the column is at minimum 14 MB just for that one column on disk, before we even add any per-row overhead.

Dictionary encoding instead builds a small lookup table at the start of the column chunk: `0=FF, 1=SL, 2=CH, 3=CU, 4=FC, ...` For the column data itself, it stores the integer index into the dictionary instead of the string. Now each value takes 1 byte (an integer 0–14) instead of 2 to 3 bytes (a string). And, crucially, that column of integers compresses far better than a column of strings, because the integers cluster (consecutive pitches often share a pitcher and pitch mix, so you see long runs of the same integer).

```
// what's actually on disk inside the column chunk:

[dictionary page]
  0 -> "FF"
  1 -> "SL"
  2 -> "CH"
  3 -> "CU"
  ...

[data pages]
  0, 0, 1, 0, 2, 0, 0, 1, 1, 0, 3, 0, 0, ...   // integer indices, ~1 byte each
```

Parquet decides dynamically whether to apply dictionary encoding. If a column has too many distinct values for the dictionary to fit in a single page, it falls back to plain encoding. You do not have to configure this. But you do have to know it is happening, so you understand why `pitch_type` is tiny on disk and `game_pk` (a unique-per-game identifier) is not.

Other encodings, briefly

Parquet supports a small fleet of other encodings, each suited to different data shapes.

Run-length encoding (RLE) stores repeated values as (value, count) pairs. `0, 0, 0, 0, 0, 1, 1, 1` becomes `(0, 5), (1, 3)`. Excellent for sorted or near-sorted columns. The `year` column of Statcast, sorted, compresses essentially to one (value, count) pair per year.

Bit packing stores small integers using only as many bits as needed. If a dictionary has 15 values, you need only 4 bits per index, not a full byte. Eight indices pack into 4 bytes instead of 8. Combined with dictionary encoding, this is where Parquet's compactness comes from.

Delta encoding stores differences between consecutive values. Useful for monotonic columns like timestamps or auto-incrementing IDs, where the differences are much smaller than the values themselves.

You will rarely choose these manually; Parquet writers pick reasonable encodings by default. But knowing they exist explains why a 7-million-row, 80-column Statcast season can compress from ~5 GB of raw CSV to under 1 GB of Parquet without anyone seeming to do anything special.

So what does this mean for us

Two takeaways from this lecture.

1. **Default to Parquet for analytical data.** CSV is for interchange with humans and legacy systems. Parquet is for storage and computation.
2. **The file format is a first-class data decision.** The same query, on the same data, can vary by an order of magnitude in cost based on how the file is laid out. Lab 3 will have you measure this on Statcast and write up the result.

Compression, splittability, and the *small-files problem*

We have agreed to use Parquet. Lets close out this unit by talking about the two final knobs that bind: how to compress the data inside the Parquet files, and how to size the files themselves. We will end on the operational mistake that breaks more young big-data deployments than any other.

Compression codecs

Parquet supports several compression codecs at the page level. The ones you will see in practice:

Codec	Compress speed	Ratio	When to use
Snappy	~250 MB/s/core	~2–3×	Default. Hot data, frequent reads. Pays for itself almost always.
LZ4	~400 MB/s/core	~2×	CPU-bound jobs where the disk is fast (NVMe, S3).
Zstd (level 3)	~150 MB/s/core	~3–4×	Best general tradeoff in 2026. Beating snappy on ratio at competitive speed.
Zstd (level 19)	~10 MB/s/core	~5×	Cold archival data. Write once, read rarely, store for years.
Gzip	~50 MB/s/core	~3–5×	Legacy. Use zstd instead in new code.

The instinct from single-machine work is "use the best compression ratio." In a distributed system, that instinct is usually wrong. Compression is not free. Every byte you save on disk costs you CPU on every read. Your read path is now:

```
disk -> decompress -> parse -> compute
```

If `decompress` is slow, you have made your reads slower, not faster, because Statcast at the scales we care about will be read many more times than it is written. Snappy is the default for a reason: it is fast enough to decompress that the disk (or the network, for S3) goes back to being the bottleneck, which is what you want. Spending more CPU to save more bytes only makes sense when CPU is cheap relative to disk or network bandwidth, which is the cold-data case.

The rule of thumb that holds up: use snappy or zstd-3 for hot data; use zstd-19 or similar for cold archival data; use lz4 only when you have measured and confirmed you are CPU-bound on decompression. Do not pick gzip in 2026 unless you are constrained to it for compatibility.

Splittability matters more than it seems

There is one more property to watch with compression codecs: **splittability**. A codec is splittable if a Spark worker can start decompressing the file from somewhere in the middle, not just from byte 0.

The classic disaster goes like this. Someone takes a 10 GB CSV file and compresses it with gzip. They put it in HDFS or S3 and ask Spark to query it. Spark sees the file and tries to assign tasks to read it. But gzip on a raw text stream is not splittable: the decompressor's internal state at byte N depends on every byte before it. So exactly one Spark task can read this file, because only one task can start the decompressor at byte 0. Your 100-core cluster reads that 10 GB file with 1 core. The job that should have taken seconds takes hours, and the engineer doesn't understand why because the data and the cluster are both fine.

Most students do not realize this until they have shipped it. Now you do.

Inside Parquet, compression is applied at the page level, not the file level. Every Parquet file is splittable regardless of what codec is used inside it, because the row groups carry their own boundaries and the decompressor restarts at each page. This is one of several reasons Parquet is the default. You stop having to think about splittability the moment you stop using text formats.

File sizing as an data-engineering decision

How big should each Parquet file be? You have already seen the answer in pieces; lets put it together.

If files are too small, the overhead of opening each file dominates the work of reading it. Every file requires a separate footer read, a separate task scheduling, a separate metadata lookup. On S3, every file is a separate HTTP request with a 50–100 ms first-byte latency. On HDFS, every file consumes NameNode memory.

If files are too large, you lose the parallelism of multiple files. A single 100 GB Parquet file with one big row group is processed by one task. A 100 GB file split into 1,000 row groups is processed by 1,000 tasks, but those 1,000 tasks still came out of one file (and so were assigned to one reader to start). Two 50 GB files give you natural parallelism right at the file boundary, where it's easiest to exploit.

The rules of thumb that hold up in practice:

- Parquet files should be roughly 128 MB to 1 GB each.
- Row groups inside the file should be 64 MB to 512 MB.
- The number of files should be roughly the level of parallelism you want for that dataset, not the number of seconds of data you have.

The small-files problem

Now the operational sin we have been building toward. It is the most common mistake we see in young big-data deployments, and you should be able to spot it in the wild.

Imagine MIDN A is generating one Parquet file per minute as Statcast pitch data streams in. This feels like a reasonable thing to do: each file is small, easy to write, easy to reason about. After a year, MIDN A has 525,600 Parquet files. Each one is roughly 50 KB. The total dataset is a hair over 25 GB.

So far so good?

No. Things have gone wrong in two places.

First, on the storage layer. HDFS holds metadata for every file in the NameNode's memory. Each file uses ~150 bytes of NameNode heap; each block uses another ~150 bytes; with replication factor 3, every file consumes around 600 bytes of NameNode RAM. 525,600 files is ~315 MB of NameNode heap *for one dataset*. Multiply by a few hundred datasets and your NameNode is over its memory budget. Object storage doesn't have this exact problem, but it has its analog: LIST operations get linearly slower as the number of objects grows, and the per-request cost adds up. A directory with 525,600 objects takes minutes to list.

Second, on the compute layer. Spark assigns at least one task per file by default. 525,600 tasks have significant overhead even if each one takes milliseconds. There is task scheduling overhead (a few ms per task). There is serialization overhead (the executor has to be told what to do for each task). There is network setup overhead (each S3 read is a fresh HTTP connection). The job that should have taken 30 seconds takes 30 minutes because it is paying scheduling overhead per file. Worse, the executors spend most of their time waiting for the next task to be assigned rather than doing useful work, so adding more workers doesn't help much.

This is the **small-files problem**. The right answer is not subtle: do not write tiny files.

Compaction: the cure

Real streaming ingestion will produce small files. You can't always avoid it. The cure is **compaction**: a periodic background job that rewrites many small files into fewer big ones. The big files are then queryable; the small files get deleted once the compaction is committed.

Conceptually:

```
# minutely files appearing in the raw zone
s3://bucket/raw/year=2024/month=06/day=15/hour=14/minute=00/pitches.parquet  (50 KB)
s3://bucket/raw/year=2024/month=06/day=15/hour=14/minute=01/pitches.parquet  (50 KB)
...
s3://bucket/raw/year=2024/month=06/day=15/hour=14/minute=59/pitches.parquet  (50 KB)

# nightly compaction job rewrites a day's worth into one big file
s3://bucket/compacted/year=2024/month=06/day=15/pitches.parquet              (~70 MB)
```

This is exactly the job that Delta Lake, Iceberg, and Hudi automate for you. Each of them has a built-in `OPTIMIZE` or compaction operation that, given a table of many small files, rewrites them into fewer large ones and atomically updates the metadata to point to the new files. You schedule the compaction nightly or hourly, and your hot queries never have to know about the underlying small files at all.

If you are working in raw Parquet on raw S3 without a lakehouse layer, you write the compaction job yourself. It's not difficult; it's just one more thing to maintain.

Takeaways from this unit

We have a mental model for distributed storage. We can place blocks on nodes. We can replicate them across racks. We can put them in object storage and decouple compute from storage. We can pick a consistency posture for our system (PA, PC, EL, EC) and read the PACELC of any system we encounter. We can choose a file format that respects analytic access patterns. We can size the files so the compute layer can actually use them.

Notice what we have *not* done. We have not run a single distributed computation. We have a place to put the data and we know what shape it should be in, but we still need an engine that can read it and do work on it across the cluster. That is the topic for Unit 2.